Project 3 - POSIX Thread Programming

Drew Houchens

Team name: Drew Houchens Team

Name of the VM: HouchensVM

Password: Broncos4me05-

Team members names:

Drew Houchens | Sole contributor: designed, implemented, tested, and documented all three tasks

Description of Project: The purpose of this project is to explore POSIX thread (pthread) programming through three practical problems. Task 1 parallelizes a substring search across multiple threads by partitioning the search index space. Task 2 implements a classic producer-consumer problem using condition variables and a circular buffer. Task 3 optimizes a multi-threaded linked list builder by reducing lock contention through local batching, switching from busy-wait spinning to blocking locks, and removing unnecessary CPU pinning.

Pseudocode:

--- Task 1: Parallel Substring Search ---

```
main:
    parse num_threads from command line
    call readf() to load s1 and s2 from strings.txt
    compute num_positions = len(s1) - len(s2) + 1
    divide [0, num_positions) into num_threads equal ranges

    for each thread i:
        assign range [start_i, end_i)
        create thread running count_substring(start_i, end_i)

    for each thread i:
        join thread i

    total = sum of all threads' local_count values
    print total

count_substring(start, end):
    local_count = 0
    for i = start to end - 1:
```

```
        if s1[i .. i+len(s2)-1] matches s2:
            local_count++
    return local_count
```

--- Task 2: Producer-Consumer ---

```
shared:
    circular buffer[15], count=0, head=0, tail=0, done=0
    mutex, cond_not_full, cond_not_empty

producer:
    open message.txt
    for each character ch in file:
        lock mutex
        while buffer is full:
            wait on cond_not_full
        buffer[tail] = ch
        tail = (tail + 1) % 15
        count++
        signal cond_not_empty
        unlock mutex
    lock mutex
```

```
    done = 1
    signal cond_not_empty
    unlock mutex

consumer:
    loop:
        lock mutex
        while buffer is empty AND not done:
            wait on cond_not_empty
        if buffer is empty AND done:
            unlock mutex
            exit loop
        ch = buffer[head]
        head = (head + 1) % 15
        count--
        signal cond_not_full
        unlock mutex
        print ch
```

--- Task 3: Optimized List-Forming ---

producer_thread:

Phase 1 (no lock):

local_head = local_tail = NULL

repeat K times:

allocate a new node

append node to local list (local_head/local_tail)

Phase 2 (one lock):

lock mutex

splice local list onto end of global List

unlock mutex

main:

parse num_threads from command line

initialize global List and mutex

record start time

create num_threads threads running producer_thread

join all threads

record end time

free all nodes

print elapsed time in microseconds

Conclusion: The programs largely worked as intended after the initial implementation, though testing on the VM revealed a few points worth adjusting. Task 1 and Task 2 compiled and produced correct output on the first run. Task 1 correctly counted 4 occurrences of "ab" in the sample string, matching the sequential version. Task 2 faithfully reproduced the contents of message.txt character by character through the circular buffer. The main challenge was thinking carefully about the termination logic in Task 2. The consumer must check whether the buffer is empty and whether the producer has finished. Getting the order of those checks wrong would cause either a deadlock (consumer sleeps forever) or lost characters (consumer exits too early). Understanding why the original code was slow required understanding concepts at the hardware level. This includes what happens inside a CPU cache when two cores fight over the same memory address, and why spinning in a tight loop wastes cycles that could be used by the thread that holds the lock. The local batching optimization was satisfying because it reduced the problem from 12,800 lock acquisitions (with 64 threads) down to just 64. If the project were to be improved, it would benefit from more extensive automated benchmarking scripts to systematically test many combinations of K and num_threads and produce graphs, rather than running each combination manually.

Lessons Learned: This project reinforced several core operating systems concepts through hands-on implementation. The most significant lesson came from Task 3, where I learned that the way you structure access to shared data matters far more than small code-level optimizations. The original list-forming code acquired a lock for every single node insertion. By restructuring the algorithm so that each thread builds a private list first and only locks once at the end, the total number of lock operations dropped from K-per-thread to exactly one per thread. I learned to minimize time spent in critical sections and how often you enter them. From Task 1, I learned the importance of partitioning the task rather than the data. If you split the string among threads, substrings that span the split boundary get missed. By instead splitting the iteration space (which indices to check), every thread reads the full shared string but only checks its assigned range. This also demonstrated that when threads only read shared data and write to private memory, you can avoid mutexes entirely. From Task 2, I gained an understanding of condition variables, which are the standard mechanism for "wait until a condition is true" in concurrent programs. These concepts apply directly to software engineering careers. Any backend system, game engine, or embedded application that uses multithreading relies on these same primitives. Understanding how lock contention and cache coherence affect performance at the hardware level is what separates writing code that works from writing code that scales.